

HƯỚNG DẪN PHÁT TRIỂN ỨNG DỤNG QUẢN LÝ TRÊN WPF VỚI ENTITY VÀ CODE FIRST

Giới thiệu

Tài liệu này cung cấp hướng dẫn chi tiết về quy trình phát triển một ứng dụng desktop sử dụng Windows Presentation Foundation (WPF) trên nền tảng .NET Framework. Mục tiêu chính là minh họa cách áp dụng phương pháp **Code-First** của **Entity Framework** để làm việc với cơ sở dữ liệu SQL Server, bao gồm việc định nghĩa mô hình dữ liệu, thiết lập quan hệ và xử lý các thao tác dữ liệu cơ bản (CRUD).

Dự án minh họa được sử dụng xuyên suốt tài liệu là một ứng dụng "**Quản lý Sinh viên**", qua đó chúng ta sẽ triển khai các kỹ thuật validation dữ liệu và xây dựng cơ sở dữ liệu quan hệ.

THÔNG TIN SINH VIÊN

Họ và Tên:
Nguyễn Văn A

Ngày sinh:
15/10/2002

Lớp học:
20T1

Thêm

Sửa

Xóa

ID	Họ và Tên	Ngày sinh	Lớp
1	Nguyễn Văn A	15/10/2002	20T1
2	Trần Thị B	20/05/2002	20T2
3	Lê Văn C	01/01/2002	20T1

1. Yêu cầu Môi trường

Để thực hiện theo hướng dẫn, môi trường phát triển cần đáp ứng các yêu cầu sau:

- **Visual Studio** (2019 hoặc mới hơn) đã được cài đặt.
- Workload **.NET desktop development** đã được kích hoạt (bao gồm WPF).
- Workload **Data storage and processing** đã được kích hoạt (bao gồm SQL Server Express LocalDB).

2. Khởi tạo Dự án và Cài đặt Entity Framework

2.1. Tạo Dự án WPF

Đầu tiên, chúng ta cần tạo một project WPF App trên nền tảng .NET Framework trong Visual Studio

và đặt tên là **StudentManagementWPF**.

2.2. Cài đặt Entity Framework

Entity Framework là một Object-Relational Mapper (ORM) giúp chúng ta tương tác với cơ sở dữ liệu thông qua các đối tượng C#. Chúng ta sẽ cài đặt nó thông qua NuGet.

1. Mở **Package Manager Console** (Tools > NuGet Package Manager > Package Manager Console).
2. Thực thi lệnh sau:
Lệnh này sẽ tải và cài đặt phiên bản ổn định mới nhất của Entity Framework 6.x
Install-Package EntityFramework

3. Thiết kế Mô hình Dữ liệu (Models)

Với phương pháp Code-First, chúng ta định nghĩa cấu trúc cơ sở dữ liệu bằng các lớp C# (POCO classes). Entity Framework sẽ dựa vào các lớp này để tự động sinh ra các bảng trong CSDL.

3.1. Model LopHoc

Tạo thư mục **Models** trong project. Sau đó, tạo file **LopHoc.cs**.

```
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;

namespace StudentManagementWPF.Models
{
    // Lớp này sẽ được ánh xạ thành bảng 'LopHocs' trong CSDL.
    public class LopHoc
    {
        [Key] // Data Annotation này chỉ định 'LopHocId' là khóa chính (Primary Key).
        public int LopHocId { get; set; }

        // Yêu cầu trường này không được để trống. ErrorMessage là thông báo lỗi khi vi phạm.
        [Required(ErrorMessage = "Tên lớp không được để trống")]
        // Giới hạn độ dài tối đa của chuỗi Tên lớp là 50 ký tự.
        [MaxLength(50)]
        public string TenLop { get; set; }

        // Navigation Property: Thể hiện mối quan hệ "một-nhiều".
        // Một LopHoc có thể chứa một tập hợp (Collection) các đối tượng Student.
        // Từ 'virtual' cho phép EF kích hoạt tính năng lazy loading.
        public virtual ICollection<Student> Students { get; set; }
    }
}
```

3.2. Model Student

Tạo file **Student.cs** trong thư mục Models và định nghĩa mối quan hệ với LopHoc.

```
using System;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema; // Cần thiết cho [ForeignKey]

namespace StudentManagementWPF.Models
{
    // Lớp này sẽ được ánh xạ thành bảng 'Students' trong CSDL.
    public class Student
    {
        [Key] // Chỉ định StudentId là khóa chính.
        public int StudentId { get; set; }

        [Display(Name = "Họ và Tên")] // Tên hiển thị cho thuộc tính (hữu ích trong các framework khác như MVC).
        [Required(ErrorMessage = "Họ và tên không được để trống.")]
        [StringLength(100, ErrorMessage = "Họ và tên không được vượt quá 100 ký tự.")]
        public string FullName { get; set; }

        [Required(ErrorMessage = "Ngày sinh không được để trống.")]
        [DataType(DataType.Date)] // Chỉ định kiểu dữ liệu là ngày tháng, không bao gồm thời gian.
        public DateTime DateOfBirth { get; set; }

        // Thuộc tính này sẽ là cột khóa ngoại (Foreign Key) trong bảng 'Students'.
        // Dấu '?' (int?) cho phép giá trị này có thể là NULL, nghĩa là một sinh viên có thể không thuộc lớp nào.
        public int? LopHocId { get; set; }

        // Navigation Property: Thể hiện "phía một" của mối quan hệ.
        // [ForeignKey("LopHocId")] chỉ định rõ rằng property này liên kết với khóa ngoại LopHocId ở trên.
        // Từ một đối tượng Student, chúng ta có thể truy cập trực tiếp đối tượng LopHoc liên quan.
        [ForeignKey("LopHocId")]
        public virtual LopHoc LopHoc { get; set; }
    }
}
```

4. Lớp Ngõ cảnh Dữ liệu (DbContext)

DbContext là một lớp trung tâm, đóng vai trò là cầu nối giữa các model và cơ sở dữ liệu. Nó quản lý kết nối, theo dõi thay đổi và thực hiện các thao tác truy vấn.

1. Tạo thư mục **Data** trong project.
2. Tạo file **StudentContext.cs** trong thư mục **Data**.

```
using StudentManagementWPF.Models;
using System.Data.Entity;

namespace StudentManagementWPF.Data
{
    // Lớp StudentContext kế thừa từ DbContext của Entity Framework.
    public class StudentContext : DbContext
    {
        // Constructor gọi đến constructor của lớp base và truyền vào tên của chuỗi kết nối.
        // EF sẽ tìm chuỗi kết nối có tên 'StudentDbConnectionString' trong file App.config.
        public StudentContext() : base("name=StudentDbConnectionString")
        {
        }

        // Mỗi thuộc tính DbSet<T> tương ứng với một bảng trong cơ sở dữ liệu.
        // Tên của thuộc tính (ví dụ: Students) sẽ được EF sử dụng làm tên bảng mặc định.
        public DbSet<Student> Students { get; set; }
        public DbSet<LopHoc> LopHocs { get; set; }
    }
}
```

3. Mở file App.config và thêm chuỗi kết nối vào trong thẻ <configuration>:

```
<connectionStrings>
  <add name="StudentDbConnectionString"
    connectionString="Data Source=(localdb)\MSSQLLocalDB;Initial
Catalog=StudentManagementDB;Integrated Security=True;MultipleActiveResultSets=True"
    providerName="System.Data.SqlClient" />
</connectionStrings>
```

5. Khởi tạo và Cập nhật CSDL với Migrations

Code First Migrations là công cụ cho phép chúng ta tạo và cập nhật schema của cơ sở dữ liệu một cách tuần tự dựa trên những thay đổi của các lớp Model.

1. Trong **Package Manager Console**, thực thi lệnh để kích hoạt Migrations:
 # Lệnh này chỉ cần chạy một lần duy nhất cho mỗi project.
 # Nó sẽ tạo ra một thư mục 'Migrations' và file cấu hình 'Configuration.cs'.
Enable-Migrations
2. Tạo một migration mới. Lệnh này sẽ phân tích các thay đổi trong Models kể từ lần migration

cuối cùng và tạo ra một file script C# mô tả các thay đổi đó.

'AddClassRelationship' là tên của migration, chúng ta nên đặt tên có ý nghĩa.

Add-Migration AddClassRelationship

3. Áp dụng các thay đổi vào CSDL. Lệnh này sẽ thực thi tất cả các migration chưa được áp dụng.

Nếu CSDL chưa tồn tại, lệnh này sẽ tạo mới CSDL và các bảng.

Nếu CSDL đã tồn tại, nó sẽ chỉ cập nhật schema.

Update-Database

6. Thiết kế Giao diện Người dùng (XAML)

Mở file **MainWindow.xaml** và thiết kế giao diện gồm khu vực nhập liệu và lưới hiển thị dữ liệu.

```
<Window x:Class="StudentManagementWPF.MainWindow"
```

```
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"(http://schemas.microsoft.com/winfx/2006/xaml/presentation)"
```

```
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"(http://schemas.microsoft.com/winfx/2006/xaml)"
```

```
mc:Ignorable="d"
```

```
Title="Quản lý Sinh viên" Height="450" Width="800" Loaded="Window_Loaded">
```

```
<Grid Margin="10">
```

```
<!-- Chia cửa sổ thành 2 hàng: hàng trên cho tiêu đề, hàng dưới chiếm phần còn lại -->
```

```
<Grid.RowDefinitions>
```

```
<RowDefinition Height="Auto"/>
```

```
<RowDefinition Height="*/"/>
```

```
</Grid.RowDefinitions>
```

```
<!-- Chia cửa sổ thành 2 cột: cột trái cho form, cột phải cho DataGrid -->
```

```
<Grid.ColumnDefinitions>
```

```
<ColumnDefinition Width="280"/>
```

```
<ColumnDefinition Width="*/"/>
```

```
</Grid.ColumnDefinitions>
```

```
<!-- Form nhập liệu đặt trong một StackPanel để các control xếp chồng lên nhau -->
```

```
<StackPanel Grid.Row="1" Grid.Column="0" Margin="0,10,10,0">
```

```
<Label Content="Họ và Tên:"/>
```

```
<TextBox x:Name="txtFullName" Margin="0,0,0,10"/>
```

```
<Label Content="Ngày sinh:"/>
```

```
<DatePicker x:Name="dpDateOfBirth" Margin="0,0,0,10"/>
```

```
<Label Content="Lớp học:"/>
```

```
<!-- ComboBox để chọn lớp học -->
```

```
<ComboBox x:Name="cmbLopHoc" Margin="0,0,0,10"
```

```
DisplayMemberPath="TenLop" <!-- Thuộc tính được hiển thị cho người dùng -->
```

```

        SelectedValuePath="LopHocId"/> <!-- Thuộc tính được dùng làm giá trị thực khi chọn -->
    >

    <StackPanel Orientation="Horizontal" HorizontalAlignment="Center" Margin="0,10,0,0">
        <Button x:Name="btnAdd" Content="Thêm" Width="70" Margin="5"
Click="btnAdd_Click"/>
        <Button x:Name="btnUpdate" Content="Sửa" Width="70" Margin="5"
Click="btnUpdate_Click"/>
        <Button x:Name="btnDelete" Content="Xóa" Width="70" Margin="5"
Click="btnDelete_Click"/>
    </StackPanel>
</StackPanel>

<!-- DataGrid để hiển thị danh sách sinh viên -->
<DataGrid x:Name="dgStudents" Grid.Row="1" Grid.Column="1" Margin="10,10,0,0"
    AutoGenerateColumns="False" IsReadOnly="True"
    SelectionChanged="dgStudents_SelectionChanged">
    <DataGrid.Columns>
        <DataGridTextColumn Header="ID" Binding="{Binding StudentId}" Width="50"/>
        <DataGridTextColumn Header="Họ và Tên" Binding="{Binding FullName}" Width="*"/>
        <DataGridTextColumn Header="Ngày sinh" Binding="{Binding DateOfBirth,
StringFormat='dd/MM/yyyy'}" Width="120"/>
        <!-- Binding đến navigation property để hiển thị tên lớp. EF sẽ tự động join bảng. -->
        <DataGridTextColumn Header="Lớp" Binding="{Binding LopHoc.TenLop}" Width="100"/>
    </DataGrid.Columns>
</DataGrid>

    <Label Grid.Row="0" Grid.Column="0" Grid.ColumnSpan="2" Content="THÔNG TIN SINH VIÊN"
FontWeight="Bold" FontSize="20" HorizontalAlignment="Center"/>
</Grid>
</Window>

```

7. Lập trình Logic Nghiệp vụ (Code-Behind)

Mở file **MainWindow.xaml.cs** để viết mã xử lý các sự kiện từ giao diện và tương tác với cơ sở dữ liệu.

```

using StudentManagementWPF.Data;
using StudentManagementWPF.Models;
using System;
using System.Data.Entity; // Cần thiết cho phương thức .Include()
using System.Linq;
using System.Windows;
using System.Windows.Controls;

```

```

namespace StudentManagementWPF
{
    public partial class MainWindow : Window
    {
        // Khởi tạo một instance của DbContext. Instance này đại diện cho một session với CSDL,
        // cho phép chúng ta truy vấn và lưu dữ liệu. Nó nên được duy trì trong suốt vòng đời của cửa
        sổ.
        private StudentContext _context = new StudentContext();

        // Biến lưu trữ đối tượng sinh viên đang được chọn trên DataGrid.
        private Student _selectedStudent = null;

        public MainWindow()
        {
            InitializeComponent();
        }

        // Sự kiện này được kích hoạt khi cửa sổ được tải lần đầu tiên.
        private void Window_Loaded(object sender, RoutedEventArgs e)
        {
            LoadClasses();
            LoadStudents();
        }

        // Tải danh sách các lớp học từ CSDL và gán làm nguồn dữ liệu cho ComboBox.
        private void LoadClasses()
        {
            cmbLopHoc.ItemsSource = _context.LopHocs.ToList();
        }

        // Tải danh sách sinh viên.
        private void LoadStudents()
        {
            // Sử dụng Eager Loading với .Include() để tải kèm thông tin LopHoc liên quan.
            // Điều này giúp tránh các lỗi liên quan đến lazy loading và tối ưu hóa số lượng truy vấn đến
            CSDL.
            dgStudents.ItemsSource = _context.Students.Include(s => s.LopHoc).ToList();
        }

        // Xóa trắng các control trên form nhập liệu.
        private void ClearForm()
        {
            txtFullName.Clear();
            dpDateOfBirth.SelectedDate = null;
        }
    }
}

```

```

        cmbLopHoc.SelectedIndex = -1;
        _selectedStudent = null;
        dgStudents.SelectedItem = null;
    }

    // Xử lý sự kiện click nút "Thêm".
    private void btnAdd_Click(object sender, RoutedEventArgs e)
    {
        try
        {
            var newStudent = new Student
            {
                FullName = txtFullName.Text,
                DateOfBirth = dpDateOfBirth.SelectedDate ?? throw new
ArgumentNullException(nameof(dpDateOfBirth), "Ngày sinh không được để trống"),
                LopHocId = (int?)cmbLopHoc.SelectedValue
            };

            _context.Students.Add(newStudent);
            _context.SaveChanges(); // Lệnh này sẽ commit các thay đổi (thêm, sửa, xóa) vào CSDL.
                                   // EF sẽ tự động kiểm tra các quy tắc validation tại đây.

            LoadStudents(); // Tải lại danh sách để hiển thị sinh viên mới.
            ClearForm();
        }
        // Bắt ngoại lệ cụ thể khi SaveChanges() thất bại do vi phạm validation.
        catch (System.Data.Entity.Validation.DbEntityValidationException dbEx)
        {
            // Trích xuất các thông báo lỗi validation chi tiết.
            var errorMessages = dbEx.EntityValidationErrors
                .SelectMany(x => x.ValidationErrors)
                .Select(x => x.ErrorMessage);

            var fullErrorMessage = string.Join("\n", errorMessages);
            MessageBox.Show("Vui lòng kiểm tra lại dữ liệu nhập:\n" + fullErrorMessage, "Lỗi
Validation", MessageBoxButton.OK, MessageBoxImage.Error);
        }
        catch (Exception ex)
        {
            MessageBox.Show("Đã có lỗi xảy ra: " + ex.Message, "Lỗi", MessageBoxButton.OK,
MessageBoxImage.Error);
        }
    }

    // Sự kiện được kích hoạt khi người dùng chọn một dòng khác trên DataGridView.

```

```

private void dgStudents_SelectionChanged(object sender, SelectionChangedEventArgs e)
{
    _selectedStudent = dgStudents.SelectedItem as Student;

    if (_selectedStudent != null)
    {
        // Hiển thị thông tin của sinh viên được chọn lên form.
        txtFullName.Text = _selectedStudent.FullName;
        dpDateOfBirth.SelectedDate = _selectedStudent.DateOfBirth;
        cmbLopHoc.SelectedValue = _selectedStudent.LopHocId;
    }
}

// Xử lý sự kiện click nút "Sửa".
private void btnUpdate_Click(object sender, RoutedEventArgs e)
{
    if (_selectedStudent == null)
    {
        MessageBox.Show("Vui lòng chọn một sinh viên để sửa.", "Thông báo",
        MessageBoxButton.OK, MessageBoxImage.Information);
        return;
    }

    try
    {
        // Cập nhật các thuộc tính của đối tượng đang được theo dõi bởi DbContext.
        _selectedStudent.FullName = txtFullName.Text;
        _selectedStudent.DateOfBirth = dpDateOfBirth.SelectedDate.Value;
        _selectedStudent.LopHocId = (int?)cmbLopHoc.SelectedValue;

        _context.SaveChanges(); // Lưu các thay đổi vào CSDL.

        LoadStudents();
        ClearForm();
    }
    catch (System.Data.Entity.Validation.DbEntityValidationException dbEx)
    {
        var errorMessages = dbEx.EntityValidationErrors.SelectMany(x =>
x.ValidationErrors).Select(x => x.ErrorMessage);
        var fullErrorMessage = string.Join("\n", errorMessages);
        MessageBox.Show("Vui lòng kiểm tra lại dữ liệu nhập:\n" + fullErrorMessage, "Lỗi
Validation", MessageBoxButton.OK, MessageBoxImage.Error);
    }
    catch (Exception ex)
    {

```

```

        MessageBox.Show("Đã có lỗi xảy ra: " + ex.Message, "Lỗi", MessageBoxButton.OK,
        MessageBoxImage.Error);
    }
}

// Xử lý sự kiện click nút "Xóa".
private void btnDelete_Click(object sender, RoutedEventArgs e)
{
    if (_selectedStudent == null)
    {
        MessageBox.Show("Vui lòng chọn một sinh viên để xóa.", "Thông báo",
        MessageBoxButton.OK, MessageBoxImage.Information);
        return;
    }

    var result = MessageBox.Show($"Bạn có chắc chắn muốn xóa sinh viên
    '{_selectedStudent.FullName}' không?", "Xác nhận xóa", MessageBoxButton.YesNo,
    MessageBoxImage.Warning);

    if (result == MessageBoxResult.Yes)
    {
        _context.Students.Remove(_selectedStudent);
        _context.SaveChanges();
        LoadStudents();
        ClearForm();
    }
}
}
}

```

8. Kết luận

Qua hướng dẫn này, chúng ta đã xây dựng thành công một ứng dụng WPF hoàn chỉnh, có khả năng thực hiện các thao tác CRUD trên một cơ sở dữ liệu quan hệ. Các kỹ thuật chính đã được áp dụng bao gồm:

- **Entity Framework Code-First:** Xây dựng CSDL từ mã nguồn C#.
- **Data Annotations:** Định nghĩa các quy tắc validation ngay trên Model.
- **Code First Migrations:** Quản lý phiên bản và cập nhật schema CSDL một cách an toàn.
- **WPF Data Binding:** Kết nối dữ liệu giữa giao diện và logic nghiệp vụ.
- **Xử lý ngoại lệ:** Bắt và hiển thị các lỗi validation một cách thân thiện với người dùng.